

Administration à Distance, Remoting WinRM et Orchestration de Parc

Ce document détaille l'ensemble des concepts architecturaux, réseau et logiciels permettant de déporter l'exécution de commandes sur un parc de serveurs distants via WinRM, d'injecter des données clientes locales, de transformer les objets à la volée et de consolider des inventaires multi-machines.

1. Architecture et Fondations du Remoting PowerShell

1.1 Le standard WS-Management et le service WinRM

Le Remoting PowerShell repose sur une architecture standardisée : le protocole **WS-Management** (*Web Services for Management*), implémenté nativement sous Windows par le service d'arrière-plan **WinRM** (*Windows Remote Management*).

Ce mécanisme encapsule l'ensemble des requêtes d'administration dans des enveloppes SOAP structurées transitant sur des ports TCP fixes et identifiés. Cette standardisation simplifie grandement l'administration en entreprise, assure une compatibilité totale avec les règles de filtrage des pare-feux et garantit la stabilité des communications à travers les routeurs et passerelles réseau.

1.2 Ports d'écoute et couches de transport

Les communications WinRM utilisent les ports normalisés par l'IANA :

- **TCP 5985 (HTTP)** : Port par défaut. Bien que le transport soit formellement du HTTP, les paquets échangés ne circulent pas en clair au sein d'un domaine Active Directory. L'ensemble de la charge utile est chiffré au niveau applicatif par le protocole d'authentification (Kerberos ou NTLM) avant d'être transmis sur le câble réseau.
- **TCP 5986 (HTTPS)** : Port sécurisé par certificat numérique TLS. Obligatoire lorsque les communications traversent des réseaux non approuvés ou lorsqu'une autorité de certification d'entreprise chiffre le flux de bout en bout.

2. Prérequis, Sécurité et Diagnostics Réseau

Préalables

2.1 Élévation de privilèges

Toute opération d'administration distante via WinRM nécessite l'exécution de la console d'administration locale avec des privilèges élevés (**Exécuter en tant qu'administrateur**). Sans cette élévation de jeton de sécurité, le moteur local refuse d'initialiser les sessions distantes ou d'interroger la configuration des démons distants.

2.2 Diagnostic réseau à double niveau (Sans requête ICMP)

Dans les architectures d'entreprise durcies, le protocole ICMP (Echo Request / Ping) est systématiquement bloqué par les pare-feux des serveurs. L'échec d'un test ICMP ne préjuge en rien de l'indisponibilité du service d'administration. Deux commandes ciblées valident la chaîne de communication :

- **Niveau Transport (Couche 4) - Test du port TCP :**
Test-NetConnection -ComputerName "SRV-FILE01" -Port 5985

La propriété booléenne `TcpTestSucceeded` confirme l'ouverture du flux réseau à travers les pare-feux intermédiaires.

- **Niveau Applicatif (Couche 7) - Test du démon WS-Management :**
Test-WSMan -ComputerName "SRV-FILE01"

Cette commande envoie une enveloppe SOAP au démon WinRM distant. La réception de la version de la pile logicielle (ex: OS: 0.0.0.0 SP: 0.0 Stack: 3.0) garantit que le service Windows écoute et accepte les requêtes de gestion.

2.3 Contexte Active Directory vs Environnement Hors-Domaine (Workgroup)

- **Au sein d'un domaine Active Directory (Scénario nominal) :**
L'authentification est déléguée au protocole Kerberos. Le contrôleur de domaine assure l'authentification mutuelle du poste client et du serveur cible. Les jetons d'identité transitent de façon chiffrée sans paramétrage préalable.
- **En environnement hors-domaine (Workgroup ou adressage IP direct) :**
Faute de contrôleur de domaine, Kerberos ne peut intervenir et NTLM prend le relais. Par mesure de sécurité, le client refuse d'envoyer ses identifiants à une machine dont il ne peut vérifier l'authenticité. Pour lever ce blocage, le client doit inscrire explicitement la cible dans sa liste d'approbation locale (*TrustedHosts*) gérée par le PSProvider WSMAN :
Consultation des autorisations actuelles
Get-Item -Path WSMAN:\localhost\Client\TrustedHosts

Autorisation explicite d'une machine ou d'une adresse IP
Set-Item -Path WSMAN:\localhost\Client\TrustedHosts -Value "192.168.144.101" -Force

3. Administration Interactive Unitaire (1-à-1) : Enter-PSSession

3.1 Objectif opérationnel

L'administration interactive un-à-un correspond au dépannage direct d'un serveur unique. Elle remplace l'ouverture lourde d'une session Bureau à Distance (RDP) pour effectuer un diagnostic, consulter un journal local ou redémarrer un service.

3.2 Syntaxe et invite modifiée (Prompt)

L'instruction ouvre un canal d'administration bidirectionnel chiffré vers la cible :

```
Enter-PSSession -ComputerName "SRV-FILE01"
```

Dès la validation de la connexion, l'invite de commande de la console locale change pour afficher le nom de la machine cible entre crochets :

```
[SRV-FILE01]: PS C:\Users\Administrateur\Documents>
```

Règle d'exécution : Dès lors que le préfixe entre crochets est visible, chaque instruction saisie s'exécute sur le processeur, dans l'espace mémoire et sur le système de fichiers du serveur distant.

3.3 Clôture de session

Pour libérer les ressources allouées dans le processus hôte du serveur distant et ramener le contexte d'exécution sur le poste local, l'opérateur doit saisir :

```
Exit-PSSession
```

L'invite de commande redevient alors locale : `PS C:\Windows\system32>`.

4. Exécution Distribuée et Parallélisme : Invoke-Command

4.1 Pourquoi passer à l'orchestration distribuée ?

Bien qu'adaptée à une intervention unitaire, Enter-PSSession ne permet pas d'automatiser des actions sur plusieurs machines simultanément.

La cmdlet Invoke-Command permet d'envoyer un bloc d'instructions (*ScriptBlock*) sur un ensemble de serveurs, sans devoir ouvrir de session interactive ni immobiliser la console

machine par machine.

4.2 Syntaxe fondamentale et exécution parallèle

Le bloc d'instructions est encapsulé entre accolades {} via le paramètre -ScriptBlock. Lorsque l'administrateur transmet un tableau de noms de serveurs sur le paramètre -ComputerName, PowerShell n'exécute pas les requêtes de manière séquentielle, mais traite l'ensemble des cibles en parallèle :

```
$Serveurs = @("SRV-DC01", "SRV-FILE01", "SRV-WEB01")
```

```
Invoke-Command -ComputerName $Serveurs -ScriptBlock {  
    Get-Service -Name Spooler  
}
```

Par défaut, PowerShell traite jusqu'à 32 machines simultanément. Cette capacité de parallélisation peut être modulée selon la bande passante réseau à l'aide du paramètre -ThrottleLimit.

5. Sérialisation, Désérialisation et Cycle de Vie des Objets (CLIXML)

Contrairement aux outils Unix qui s'échangent des flux textuels, PowerShell fait transiter des objets structurés. Un objet .NET complexe en mémoire ne pouvant être téléporté physiquement sur un câble réseau, PowerShell applique un mécanisme de sérialisation.

5.1 Le cycle de transit réseau

1. **Exécution locale sur la cible** : La commande s'exécute sur le serveur distant et produit des objets complets en mémoire vive (RAM).
2. **Sérialisation XML (CLIXML)** : Le moteur WinRM distant extrait l'état et la valeur des propriétés de l'objet pour les encoder sous forme de flux XML structuré.
3. **Transport** : Le flux XML transite sur le port TCP 5985.
4. **Désérialisation côté client** : Sur la console locale, PowerShell lit le flux XML et reconstruit un objet mémoire statique.

5.2 Propriétés préservées vs Méthodes neutralisées

L'objet désérialisé reçu par l'administrateur est un cliché statique (*snapshot*) de l'état du serveur au moment exact de la mesure :

- **Les propriétés de données sont conservées** : Le nom, l'identifiant PID, l'espace mémoire utilisé ou le statut d'un service restent lisibles, triables (Sort-Object) et filtrables (Where-Object).

- **Les méthodes d'action sont désactivées** : L'objet porte le type Deserialized. (ex: Deserialized.System.Diagnostics.Process). Les méthodes actives ne peuvent être appelées depuis le poste client :

Sur le client local :

```
$ProcessusDistant = Invoke-Command -ComputerName "SRV-01" -ScriptBlock { Get-Process -Name notepad }
```

ERREUR : La methode .Kill() est inexistante ou inactive sur un objet deserialise :

```
$ProcessusDistant.Kill()
```

Règle d'or : Toute action modificatrice (démarrage, arrêt, suppression) doit impérativement être codée et exécutée à l'intérieur du -ScriptBlock distant, et non tentée sur l'objet désérialisé rapatrié.

6. Injection de Variables Locales : Le Modificateur de Portée \$using:

6.1 L'isolation des espaces mémoires

Le code transmis à Invoke-Command s'exécute dans un processus distant dédié (wsmpvhost.exe) sur le serveur distant. Ce processus distant ne partage aucun espace mémoire avec le poste client de l'administrateur.

Considérons l'exemple suivant :

Variable declaree sur la console du client

```
$SeuilRAMMo = 150
```

ECHEC TECHNIQUE :

```
Invoke-Command -ComputerName "SRV-FILE01" -ScriptBlock {
    # Sur le serveur distant, $SeuilRAMMo n'existe pas : la variable vaut $null
    Get-Process | Where-Object WorkingSet -gt ($SeuilRAMMo * 1MB)
}
```

6.2 Le préfixe de portée \$using:

Pour indiquer au moteur PowerShell qu'une variable doit être résolue localement sur le poste client avant d'être transmise et injectée dans le contexte du bloc distant, il est obligatoire de préfixer la variable par \$using :

```
$SeuilRAMMo = 150
```

```
Invoke-Command -ComputerName "SRV-FILE01" -ScriptBlock {  
    # La valeur locale (150) est injectée correctement dans le processus distant  
    Get-Process | Where-Object WorkingSet -gt ($using:SeuilRAMMo * 1MB)  
}
```

7. Propriétés Calculées à la Volée (Calculated Properties)

7.1 Pourquoi transformer les grandeurs à la source ?

Certaines propriétés système renvoient des valeurs brutes difficiles à interpréter directement. Par exemple, la propriété `WorkingSet` de `Get-Process` renvoie une quantité d'octets bruts (ex: 157286400 octets pour 150 Mo).

Plutôt que de rapatrier des chiffres bruts pour les recalculer ensuite, PowerShell permet de créer des **colonnes ou propriétés calculées dynamiquement** au sein de la cmdlet `Select-Object`.

7.2 Syntaxe par table de hachage (@{Name = ...; Expression = ...})

Une propriété calculée est définie par une table de hachage (*Hashtable*) comportant obligatoirement deux clés :

- **Name (ou Label)** : Le nom de la nouvelle propriété générée.
- **Expression** : Un bloc de script {} calculant la valeur à partir de l'objet en cours de traitement (\$_).

```
Select-Object -Property Name, Id, @{Name = "RAM_Mo"; Expression = {  
[math]::Round($_.WorkingSet / 1MB, 2) }}
```

Dans cet exemple, pour chaque processus, PowerShell :

1. Divise la propriété `WorkingSet` par 1MB (soit 1 048 576 octets).
2. Applique la méthode mathématique statique `[math]::Round(valeur, 2)` pour arrondir le résultat à deux décimales.
3. Projette le résultat sous une nouvelle colonne propre nommée `RAM_Mo`.

8. Consolidation Multi-Sources et Métadonnée

PSComputerName

Lorsque l'administrateur interroge 20 serveurs simultanément via Invoke-Command, toutes les machines renvoient leurs flux d'objets sur le même pipeline client.

8.1 Traçabilité automatique via PSComputerName

Pour identifier sans ambiguïté la provenance de chaque objet sans devoir modifier le code distant, PowerShell enrichit automatiquement chaque objet désérialisé d'une propriété synthétique :

PSComputerName

Cette propriété contient le nom d'hôte de la machine sur laquelle l'objet a été capturé.

8.2 Exploitation pour le tri et le formatage

Grâce à PSComputerName, il devient possible d'agréger, de trier et de mettre en forme les données issues de l'ensemble du parc :

```
$Resultats | Select-Object -Property PSComputerName, Name, Id, RAM_Mo |  
    Sort-Object -Property RAM_Mo -Descending |  
    Format-Table -AutoSize
```

9. Architecture d'un Script d'Orchestration d'Audit

Un script d'administration en production ne doit jamais lancer d'instructions aveugles sur des machines sans validation préalable. Il repose sur un modèle d'exécution en plusieurs phases.

9.1 Déclaration des paramètres d'entrée

L'en-tête déclare les variables attendues en spécifiant leur type pour assurer la robustesse du script :

- [string[]]\$ServeursCibles : Un tableau de chaînes de caractères autorisant la saisie de plusieurs hôtes séparés par des virgules.
- [int]\$SeuilRAMMo = 150 : Un entier fixant la valeur limite avec une valeur de repli par défaut.
- [switch]\$ExporterCSV : Un commutateur binaire (\$true si présent lors de l'appel, \$false sinon).

9.2 Filtre de pré-vol (Pre-flight check)

Pour éviter que la commande globale ne soit ralentie par des serveurs éteints ou des pannes réseau, le script parcourt au préalable les cibles pour isoler les machines joignables :

```
$ServeursValides = @()
```

```
foreach ($Serveur in $ServeursCibles) {  
    if (Test-WSMan -ComputerName $Serveur -ErrorAction SilentlyContinue) {  
        $ServeursValides += $Serveur  
    } else {  
        Write-Host "ALERTE : Le serveur $Serveur ne répond pas sur WinRM." -ForegroundColor  
Red  
    }  
}
```

9.3 Collecte conditionnelle et exportation

Si le tableau \$ServeursValides contient au moins une machine, la requête globale est lancée. Si le commutateur -ExporterCSV a été fourni, les résultats sont exportés de façon persistante selon les standards d'archivage :

- Vérification et création forcée de l'arborescence (Test-Path et New-Item -ItemType Directory -Force).
- Exportation vers Export-Csv avec les trois paramètres essentiels :
 - -Delimiter ";" : Séparateur standard pour l'ouverture directe dans Microsoft Excel.
 - -NoTypeInfo : Suppression de la ligne d'en-tête de métadonnées PowerShell #TYPE....
 - -Encoding UTF8 : Préservation des caractères accentués.